

Zbirka Scilab zadataka iz laboratorijskih vježbi

Zadatak 1: Nasumični signali

Napisati Scilab program koji generiše i prikazuje:

1. nasumični signal dužine $N = 1000$ sa uniformnom raspodjelom u intervalu $[-2, 2]$,
2. nasumični signal dužine $N = 1000$ sa normalnom raspodjelom, srednjom vrijednošću 0 i varijansom 3,
3. nasumičnu sinusoidalnu sekvencu dužine 31 oblika

$$x[n] = A \cos(\omega_0 n + \varphi),$$

gdje je $\omega_0 = 0.2\pi$, $0 \leq A \leq 4$ i $0 \leq \varphi \leq 2\pi$.

Rješenje

```
clf;

// Broj uzoraka
N = 1000;
n = 0:N-1;

// 1. Uniformni signal u intervalu [-2, 2]
// rand(1,N) daje vrijednosti iz [0,1]
// transformacija 4*rand(1,N)-2 daje interval [-2,2]
x_uniform = 4 * rand(1, N) - 2;

subplot(3,1,1);
plot2d3(n, x_uniform);
xlabel("Vrijeme n");
ylabel("Amplituda");
title("Uniformni nasumicni signal u intervalu [-2,2]");

// 2. Normalni signal sa varijansom 3
// rand(1,N,"normal") ima varijansu 1
// mnozenjem sa sqrt(3) dobija se varijansa 3
x_normal = sqrt(3) * rand(1, N, "normal");

subplot(3,1,2);
plot2d3(n, x_normal);
xlabel("Vrijeme n");
```

```

ylabel("Amplituda");
title("Normalni nasumicni signal, varijansa 3");

// 3. Nasumicna sinusoidalna sekvenca
n2 = 0:30;

omega0 = 0.2 * %pi;
A = 4 * rand();
phi = 2 * %pi * rand();

x_sin = A * cos(omega0 * n2 + phi);

subplot(3,1,3);
plot2d3(n2, x_sin);
scatter(n2, x_sin, ".");
xlabel("Vrijeme n");
ylabel("Amplituda");
title("Nasumicna sinusoidalna sekvenca");

```

Objašnjenje: Funkcija `rand` se koristi za generisanje nasumičnih vrijednosti. Za promjenu intervala koristi se linearna transformacija, a za promjenu varijanse kod normalne raspodjele signal se množi sa standardnom devijacijom.

Zadatak 2: Eksponencijalna sekvenca i energija

Napisati Scilab program koji generiše sekvenču

$$x[n] = 2(0.8)^n, \quad 0 \leq n \leq 20.$$

Program treba da:

1. prikaže sekvenču kao diskretni signal,
2. izračuna energiju sekvenče,
3. ispiše energiju na ekran.

Rješenje

```
clf;

// Indeksi sekvenče
n = 0:20;

// Eksponencijalna sekvenca
x = 2 * (0.8 .^ n);

// Prikaz sekvenče
plot2d3(n, x);
scatter(n, x, ".");

xlabel("Vrijeme n");
ylabel("Amplituda");
title("Eksponencijalna sekvenca x[n] = 2*(0.8)^n");

// Racunanje energije
E = sum(abs(x).^2);

disp("Energija sekvenče je:");
disp(E);
```

Objašnjenje: Operator `.^` koristi se zato što se stepenovanje vrši element po element. Energija se računa kao suma kvadrata amplituda svih uzoraka.

Zadatak 3: Sinusoidalna sekvenca i snaga

Napisati Scilab program koji generiše signal

$$x[n] = 3 \cos(0.2\pi n), \quad 0 \leq n \leq 40.$$

Program treba da:

1. prikaže signal kao diskretnu sekvenču,
2. postavi labele i naslov grafa,
3. izračuna prosječnu snagu signala na jednom periodu.

Rješenje

```
clf;

// Indeksi
n = 0:40;

// Parametri sinusoidalne sekvence
A = 3;
omega0 = 0.2 * %pi;

// Generisanje signala
x = A * cos(omega0 * n);

// Prikaz signala
plot2d3(n, x);
scatter(n, x, ".");

xlabel("Vrijeme n");
ylabel("Amplituda");
title("Sinusoidalna sekvenca x[n] = 3cos(0.2*pi*n)");
mtlb_grid;

// Osnovni period je N = 10
N = 10;

// Racunanje prosjecne snage na jednom periodu
P = (1/N) * sum(abs(x(1:N)).^2);

disp("Prosjecna snaga signala je:");
disp(P);
```

Objašnjenje: Signal je periodičan, a za $\omega_0 = 0.2\pi$ osnovni period je $N = 10$. Snaga se računa kao srednja vrijednost kvadrata uzoraka u jednom periodu.

Zadatak 4: Moving average filter

Napisati Scilab program koji generiše ulazni signal

$$x[n] = \cos(2\pi \cdot 0.05n) + \cos(2\pi \cdot 0.45n), \quad 0 \leq n \leq 100.$$

Zatim na signal primijeniti moving average filter u $M = 5$ tačaka:

$$y[n] = \frac{1}{5} (x[n] + x[n-1] + x[n-2] + x[n-3] + x[n-4]).$$

Program treba prikazati ulazni i izlazni signal.

Rješenje

```
clf;

// Indeksi
n = 0:100;

// Komponenta niske frekvencije
s1 = cos((2*pi)*0.05*n);

// Komponenta visoke frekvencije
s2 = cos((2*pi)*0.45*n);

// Ulazni signal
x = s1 + s2;

// Moving average filter
M = 5;
num = ones(1, M);
den = 1;

// Filtriranje
y = filter(num, den, x) / M;

// Prikaz ulaznog signala
subplot(2,1,1);
plot(n, x);
xlabel("Vrijeme n");
ylabel("Amplituda");
title("Ulazni signal x[n]");

// Prikaz izlaznog signala
subplot(2,1,2);
plot(n, y);
xlabel("Vrijeme n");
ylabel("Amplituda");
title("Izlazni signal nakon moving average filtera");
```

Objašnjenje: Moving average filter usrednjava susjedne uzorke. Zbog toga više potiskuje visokofrekventnu komponentu signala.

Zadatak 5: Provjera linearnosti sistema u Scilab-u

Dat je sistem

$$y[n] = x[n]x[n-1].$$

Napisati Scilab program koji provjerava linearnost sistema koristeći dvije ulazne sekvence:

$$x_1[n] = \cos(2\pi \cdot 0.1n),$$

$$x_2[n] = \cos(2\pi \cdot 0.4n).$$

Koristiti konstante $a = 2$ i $b = -3$. Program treba izračunati:

$$x[n] = ax_1[n] + bx_2[n],$$

zatim izlaz $y[n]$ za kombinovani ulaz i signal

$$y_t[n] = ay_1[n] + by_2[n].$$

Na kraju prikazati razliku

$$d[n] = y[n] - y_t[n].$$

Rješenje

```
clf;

n = 0:40;

a = 2;
b = -3;

// Ulazne sekvence
x1 = cos(((2*pi)*0.1)*n);
x2 = cos(((2*pi)*0.4)*n);

// Kombinovani ulaz
x = a*x1 + b*x2;

// Kasnjenje za jedan uzorak
x1_delay = [0, x1(1:$-1)];
x2_delay = [0, x2(1:$-1)];
x_delay = [0, x(1:$-1)];

// Izlazi sistema y[n] = x[n]x[n-1]
y1 = x1 .* x1_delay;
y2 = x2 .* x2_delay;
y = x .* x_delay;

// Linearna kombinacija pojedinačnih izlaza
yt = a*y1 + b*y2;
```

```

// Razlika
d = y - yt;

// Prikaz
subplot(3,1,1);
plot2d3(n, y);
scatter(n, y, ".");
xlabel("Vrijeme n");
ylabel("Amplituda");
title("Izlaz y[n] za kombinovani ulaz");

subplot(3,1,2);
plot2d3(n, yt);
scatter(n, yt, ".");
xlabel("Vrijeme n");
ylabel("Amplituda");
title("yt[n] = a*y1[n] + b*y2[n]");

subplot(3,1,3);
plot2d3(n, d);
scatter(n, d, ".");
xlabel("Vrijeme n");
ylabel("Amplituda");
title("Razlika d[n] = y[n] - yt[n]");

```

Objašnjenje: Ako je sistem linearan, razlika $d[n]$ treba biti nula za sve uzorke. Ako razlika nije nula, sistem nije linearan.

Zadatak 6: Konvolucija i funkcija filter

Dat je impulsni odziv FIR sistema

$$h[n] = \{3, 2, 1, -2, 1, 0, -4, 0, 3\}$$

i ulazna sekvenca

$$x[n] = \{1, -2, 3, -4, 3, 2, 1\}.$$

Napisati Scilab program koji:

1. računa izlaz pomoću funkcije `conv`,
2. računa izlaz pomoću funkcije `filter`,
3. prikazuje oba izlaza,
4. prikazuje razliku između rezultata.

Rješenje

```
clf;

// Impulsni odziv
h = [3, 2, 1, -2, 1, 0, -4, 0, 3];

// Ulazna sekvenca
x = [1, -2, 3, -4, 3, 2, 1];

// Izlaz preko konvolucije
y_conv = conv(h, x);

// Da bi filter dao isti broj uzoraka,
// ulaz se proširuje nulama
x_padded = [x zeros(1, length(h)-1)];

// Izlaz preko filter funkcije
y_filter = filter(h, 1, x_padded);

// Indeksi za prikaz
n = 0:length(y_conv)-1;

// Prikaz izlaza dobijenog konvolucijom
subplot(3,1,1);
plot2d3(n, y_conv);
scatter(n, y_conv, ".");
xlabel("Vrijeme n");
ylabel("Amplituda");
title("Izlaz dobijen konvolucijom");

// Prikaz izlaza dobijenog filter funkcijom
subplot(3,1,2);
plot2d3(n, y_filter);
```



```

scatter(n, y_filter, ".");
xlabel("Vrijeme n");
ylabel("Amplituda");
title("Izlaz dobijen funkcijom filter");

// Razlika
d = y_conv - y_filter;

subplot(3,1,3);
plot2d3(n, d);
scatter(n, d, ".");
xlabel("Vrijeme n");
ylabel("Amplituda");
title("Razlika izmedju conv i filter rezultata");

// Ispis rezultata
disp("Izlaz dobijen funkcijom conv:");
disp(y_conv);

disp("Izlaz dobijen funkcijom filter:");
disp(y_filter);

```

Objašnjenje: Kod FIR sistema izlaz se može dobiti konvolucijom ulaza i impulsnog odziva. Funkcija `filter` daje isti rezultat ako se ulaz proširi nulama tako da se dobije puna dužina konvolucije.

Zadatak 7: DTFT i osobina vremenskog pomaka

Napisati Scilab program koji generiše sekvencu

$$x[n] = [1, 2, 3, 4, 5]$$

i njenu vremenski pomjerenu verziju za $D = 4$:

$$y[n] = x[n - D]$$

Program treba da:

1. izračuna DTFT obje sekvence pomoću funkcije **freqz**,
2. prikaže amplitudni spektar originalne i pomjerene sekvence,
3. prikaže fazni spektar originalne i pomjerene sekvence,
4. uporedi rezultate i objasni uticaj vremenskog pomaka.

Rješenje

```
clf;

// Frekvencijska osa
w = -%pi:(2*%pi)/511:%pi;

// Pomjeraj
D = 4;

// Originalna sekvenca
x = [1 2 3 4 5];

// Pomjerena sekvenca
y = [zeros(1,D), x];

// Funkcija freqz
function H = freqz(num, den, w)

    temp = %e.^(%i .* w);

    num2 = 0;

    for i = 1:length(num)
        num2 = num2 + num(i) * temp.^(-(i-1));
    end

    den2 = 0;

    for i = 1:length(den)
        den2 = den2 + den(i) * temp.^(-(i-1));
    end

    H = num2 ./ den2;
```

```

endfunction

// DTFT
X = freqz(x, 1, w);
Y = freqz(y, 1, w);

// Prikaz amplitudnih spektara
subplot(2,2,1);
plot(w/%pi, abs(X));
xlabel("omega/pi");
ylabel("Amplituda");
title("Amplitudni spektar originalne sekvence");

subplot(2,2,2);
plot(w/%pi, abs(Y));
xlabel("omega/pi");
ylabel("Amplituda");
title("Amplitudni spektar pomjerene sekvence");

// Prikaz faznih spektara
subplot(2,2,3);
plot(w/%pi, atan(imag(X), real(X)));
xlabel("omega/pi");
ylabel("Faza");
title("Fazni spektar originalne sekvence");

subplot(2,2,4);
plot(w/%pi, atan(imag(Y), real(Y)));
xlabel("omega/pi");
ylabel("Faza");
title("Fazni spektar pomjerene sekvence");

```

Objašnjenje

Vremenski pomak sekvence ne mijenja amplitudni spektar DTFT-a, ali utiče na fazni spektar. Pomjeranje za D uzoraka uvodi dodatni linearni fazni član:

$$e^{-j\omega D}$$

Zbog toga amplitudni spektar ostaje isti, dok faza postaje linearno pomjerena.

Zadatak 8: Osobina konvolucije DTFT-a

Dane su sekvence

$$x_1[n] = [1, 2, 1]$$

i

$$x_2[n] = [1, -1, 1]$$

Napisati Scilab program koji:

1. izračunava linearnu konvoluciju sekvenci,
2. računa DTFT obje sekvence,
3. računa proizvod DTFT-ova,
4. poredi DTFT konvolucije sa proizvodom DTFT-ova.

Rješenje

```
clf;

// Frekvencijska osa
w = -%pi:(2*%pi)/511:%pi;

// Sekvence
x1 = [1 2 1];
x2 = [1 -1 1];

// Konvolucija
y = conv(x1, x2);

// Funkcija freqz
function H = freqz(num, den, w)

    temp = %e.^(%i .* w);

    num2 = 0;

    for i = 1:length(num)
        num2 = num2 + num(i) * temp.^(-(i-1));
    end

    den2 = 0;

    for i = 1:length(den)
        den2 = den2 + den(i) * temp.^(-(i-1));
    end

    H = num2 ./ den2;

endfunction
```

```

// DTFT
H1 = freqz(x1, 1, w);
H2 = freqz(x2, 1, w);

// Proizvod DTFT-ova
HP = H1 .* H2;

// DTFT konvolucije
HY = freqz(y, 1, w);

// Prikaz
subplot(2,1,1);
plot(w/%pi, abs(HP));
xlabel("omega/pi");
ylabel("Amplituda");
title("Proizvod DTFT-ova");

subplot(2,1,2);
plot(w/%pi, abs(HY));
xlabel("omega/pi");
ylabel("Amplituda");
title("DTFT konvolucije");

```

Objašnjenje

Osobina konvolucije DTFT-a kaže:

$$x_1[n] * x_2[n] \Longleftrightarrow X_1(e^{j\omega})X_2(e^{j\omega})$$

To znači da konvolucija u vremenskom domenu odgovara množenju u frekventnom domenu. Zbog toga se oba dobijena spektra poklapaju.

Zadatak 9: DFT i Parsevalova relacija

Napisati Scilab program koji za sekvencu

$$x[n] = [1, 2, 3, 4, 5, 4, 3, 2]$$

treba da:

1. izračuna DFT koristeći funkciju `fft`,
2. izračuna energiju sekvence u vremenskom domenu,
3. izračuna energiju u frekventnom domenu koristeći Parsevalovu relaciju,
4. uporedi rezultate.

Rješenje

```
clf;

// Sekvenca
x = [1 2 3 4 5 4 3 2];

// Duzina sekvence
N = length(x);

// DFT
X = fft(x);

// Energija u vremenskom domenu
E1 = sum(abs(x).^2);

// Energija u frekventnom domenu
E2 = (1/N) * sum(abs(X).^2);

// Ispis
disp("Energija u vremenskom domenu:");
disp(E1);

disp("Energija u frekventnom domenu:");
disp(E2);

// Prikaz amplitudnog spektra
k = 0:N-1;

plot2d3(k, abs(X));
scatter(k, abs(X), ".");
xlabel("k");
ylabel("Amplituda");
title("Amplitudni spektar DFT-a");
```

Objašnjenje

Parsevalova relacija pokazuje da je ukupna energija signala ista u vremenskom i frekventnom domenu:

$$\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2$$

Rezultati $E1$ i $E2$ trebaju biti jednaki ili veoma bliski zbog numeričke preciznosti računara.

Zadatak 10: Cirkularna konvolucija i DFT

Dane su sekvence jednake dužine

$$g_1[n] = [1, 2, 3, 4]$$

i

$$g_2[n] = [1, -1, 1, -1]$$

Napisati Scilab program koji:

1. računa cirkularnu konvoluciju koristeći DFT,
2. računa DFT obje sekvence,
3. izračuna IDFT proizvoda DFT-ova,
4. prikaže rezultat cirkularne konvolucije.

Rješenje

```
clf;

// Sekvence
g1 = [1 2 3 4];
g2 = [1 -1 1 -1];

// DFT sekvenci
G1 = fft(g1);
G2 = fft(g2);

// Proizvod DFT-ova
Y = G1 .* G2;

// Inverzni DFT
y = real(fft(Y, 1));

// Indeksi
n = 0:length(y)-1;

// Prikaz rezultata
plot2d3(n, y);
scatter(n, y, ".");
xlabel("n");
ylabel("Amplituda");
title("Cirkularna konvolucija preko DFT-a");

// Ispis rezultata
disp("Rezultat cirkularne konvolucije:");
disp(y);
```


Objašnjenje

Kod DFT-a množenje u frekventnom domenu odgovara cirkularnoj konvoluciji u vremenskom domenu:

$$g_1[n]g_2[n] \iff G_1[k]G_2[k]$$

Zbog toga se cirkularna konvolucija može dobiti:

1. računanjem DFT-a obje sekvence,
2. množenjem njihovih DFT-ova,
3. računanjem inverznog DFT-a proizvoda.

Zadatak 11: Kaskadni filteri i amplitudni odziv (Lab 6)

Dat je jednostavan osrednjavajući (moving-average) filter opisan prenosnom funkcijom:

$$H(z) = \frac{1}{2} + \frac{1}{2}z^{-1}$$

Napisati Scilab program koji:

1. formira kaskadni sistem koji se sastoji od $K = 4$ ovakve sekcije,
2. računa frekventni odziv kaskadnog sistema na intervalu $\omega \in [0, \pi]$,
3. prikazuje amplitudni odziv (pojačanje) kaskade u decibelima (dB),
4. na grafiku jasno označava uticaj kaskadiranja na slabljenje signala.

Rješenje

```
clf;

// Frekvencijska osa
w = 0:(%pi/511):%pi;

// Prenosna funkcija jedne sekcije
num = [0.5, 0.5];
den = [1];

// Broj sekcija u kaskadi
K = 4;

// Nalazimo numerator kaskadnog sistema uzastopnom konvolucijom
num_kaskade = num;
for i = 1:(K-1)
    num_kaskade = conv(num_kaskade, num);
end

// Funkcija freqz
function H = freqz(num, den, w)
    temp = %e.^(%i .* w);
    num2 = 0;
    for i = 1:length(num)
        num2 = num2 + num(i) * temp.^(-(i-1));
    end
    den2 = 0;
    for i = 1:length(den)
        den2 = den2 + den(i) * temp.^(-(i-1));
    end
    H = num2 ./ den2;
endfunction

// Frekventni odziv jedne sekcije i kaskade
H_jedan = freqz(num, den, w);
H_kaskada = freqz(num_kaskade, den, w);
```

```
// Racunanje pojacanja u dB
Gain_jedan = 20 * log10(abs(H_jedan));
Gain_kaskada = 20 * log10(abs(H_kaskada));

// Prikaz rezultata
plot(w/%pi, Gain_jedan, 'b');
plot(w/%pi, Gain_kaskada, 'r');
xlabel("omega/pi");
ylabel("Pojacanje [dB]");
title("Magnitudni odziv kaskadnog filtera");
legend("Jedna sekcija", "Kaskada od 4 sekcije");
mtlb_grid;
```

Objašnjenje

Kaskadiranjem K istih filterskih sekcija, ukupna prenosna funkcija postaje $(H(z))^K$. U frekventnom domenu, ovo znači da se amplitudni odziv podiže na stepen K . U logaritamskoj (dB) skali, kaskadiranje dovodi do oštrijeg pada prelazne karakteristike i većeg gušenja u nepropusnom opsegu, ali i do sužavanja propusnog opsega (smanjenja 3-dB *cutoff* frekvencije).

Zadatak 12: Linearna faza FIR filtera (Lab 6)

Zadan je impulsni odziv FIR filtera:

$$h[n] = [1, -2, 3, -2, 1]$$

Napisati Scilab program koji:

1. izračunava frekventni odziv (DTFT) ovog filtera,
2. prikazuje magnitudni spektar,
3. prikazuje fazni spektar,
4. na osnovu grafa provjerava osobinu linearne faze.

Rješenje

```
clf;

// Frekvencijska osa
w = -%pi:(2*%pi)/511:%pi;

// Impulsni odziv FIR filtera
h = [1, -2, 3, -2, 1];

// Funkcija freqz
function H = freqz(num, den, w)
    temp = %e.^(%i .* w);
    num2 = 0;
    for i = 1:length(num)
        num2 = num2 + num(i) * temp.^(-(i-1));
    end
    H = num2; // den je 1 za FIR filtere
endfunction

// Racunanje DTFT-a
H = freqz(h, 1, w);

// Amplitudni spektar
subplot(2,1,1);
plot(w/%pi, abs(H));
xlabel("omega/pi");
ylabel("Amplituda");
title("Amplitudni spektar simetricnog FIR filtera");
mtlb_grid;

// Fazni spektar
subplot(2,1,2);
faza = atan(imag(H), real(H));

// Unwrapping faze za pravilan prikaz linearnosti
faza_unwrapped = unwrap(faza);
```

```
plot(w/%pi, faza_unwrapped);  
xlabel("omega/pi");  
ylabel("Faza [rad]");  
title("Fazni spektar (unwrapped)");  
mtlb_grid;
```

Objašnjenje

Zadani FIR filter ima simetričan impulsni odziv dužine $N = 5$, gdje važi $h[n] = h[N - 1 - n]$. Osobina ovakvih sistema (Tip 1 linearni fazni FIR filteri) jeste da posjeduju striktno linearnu faznu karakteristiku. Graf faze mora biti prava linija sa nagibom koji odgovara konstantnom grupnom kašnjenju od $(N - 1)/2 = 2$ uzorka.

Zadatak 13: Realna ograničenost sistema (Lab 7)

Data je prenosna funkcija IIR filtera:

$$G_1(z) = \frac{1 - 0.5z^{-1}}{1 + 0.8z^{-1} + 0.5z^{-2}}$$

Napisati Scilab program koji:

1. računa maksimalnu vrijednost magnitudnog odziva (ampMax) za sistem $G_1(z)$,
2. testira da li je $G_1(z)$ realno ograničena funkcija,
3. ukoliko nije, vrši skaliranje kako bi generisao novu prenosnu funkciju $G_2(z)$ koja je realno ograničena i ima istu magnitudnu karakteristiku,
4. grafički prikazuje i poredi amplitudne spektre $G_1(z)$ i $G_2(z)$.

Rješenje

```
clf;

// Frekvencijska osa
w = 0:(%pi/511):%pi;

// Prenosna funkcija G1(z)
num1 = [1, -0.5];
den1 = [1, 0.8, 0.5];

// Funkcija freqz
function H = freqz(num, den, w)
    temp = %e.^(%i .* w);
    num2 = 0;
    for i = 1:length(num)
        num2 = num2 + num(i) * temp.^(-(i-1));
    end
    den2 = 0;
    for i = 1:length(den)
        den2 = den2 + den(i) * temp.^(-(i-1));
    end
    H = num2 ./ den2;
endfunction

// Magnitudni odziv
H1 = freqz(num1, den1, w);
mag1 = abs(H1);

// Odredjivanje maksimalne vrijednosti
ampMax = max(mag1);

disp("Maksimalna amplituda G1(z) je:");
disp(ampMax);

// Kreiranje G2(z) koja je realno ogranicena
```

```

num2 = num1 / ampMax;

// Novi magnitudni odziv
H2 = freqz(num2, den1, w);
mag2 = abs(H2);

// Prikaz rezultata
plot(w/%pi, mag1, 'b');
plot(w/%pi, mag2, 'r--');
xlabel("omega/pi");
ylabel("Amplituda");
title("Skaliranje za postizanje realne ogranicenosti");
legend("G1(z) original", "G2(z) skaliran (Bounded Real)");
mtlb_grid;

```

Objašnjenje

Funkcija je u skupu realnih brojeva ograničena (Bounded Real - BR) ako je stabilna i ukoliko za svako ω vrijedi $|G(e^{j\omega})| \leq 1$. Ako je $\max(|G(e^{j\omega})|) > 1$, sistem se može modifikovati jednostavnim dijeljenjem numeratora sa maksimalnom detektovanom amplitudom (ampMax). Dobijeni sistem $G_2(z)$ zadržava identičan oblik frekventne karakteristike, ali mu je maksimalno pojačanje tačno 1.

Zadatak 14: Komplementarni filteri po snazi (Lab 7)

Zadana su dva jednostavna FIR filtera (niskopropusni i visokopropusni):

$$H_0(z) = \frac{1}{2} + \frac{1}{2}z^{-1}$$

$$H_1(z) = \frac{1}{2} - \frac{1}{2}z^{-1}$$

Napisati Scilab program koji:

1. računa amplitudne odzive oba filtera na opsegu $\omega \in [0, \pi]$,
2. računa sumu kvadrata njihovih amplitudnih odziva za svaku frekvenciju,
3. grafički provjerava definiciju da li su filteri komplementarni po snazi.

Rješenje

```
clf;

// Frekvencijska osa
w = 0:(%pi/511):%pi;

// Filteri
num0 = [0.5, 0.5];
num1 = [0.5, -0.5];
den = [1];

// Funkcija freqz
function H = freqz(num, den, w)
    temp = %e.^(%i .* w);
    num2 = 0;
    for i = 1:length(num)
        num2 = num2 + num(i) * temp.^(-(i-1));
    end
    H = num2;
endfunction

// Odzivi
H0 = freqz(num0, den, w);
H1 = freqz(num1, den, w);

// Suma kvadrata amplitudnih odziva
suma_snage = abs(H0).^2 + abs(H1).^2;

// Prikaz amplitudnih karakteristika
subplot(2,1,1);
plot(w/%pi, abs(H0), 'b');
plot(w/%pi, abs(H1), 'r');
xlabel("omega/pi");
ylabel("Amplituda");
title("Amplitudni odzivi H0(z) i H1(z)");
```



```

legend("H0(z) - Lowpass", "H1(z) - Highpass");
mtlb_grid;

// Prikaz sume snaga
subplot(2,1,2);
plot(w/%pi, suma_snage, 'k', "LineWidth", 2);
xlabel("omega/pi");
ylabel("Suma kvadrata amplituda");
title("Provjera komplementarnosti po snazi");
// Postavljanje y ose radi jasnijeg prikaza konstante
gca().data_bounds = [0, 0; 1, 2];
mtlb_grid;

```

Objašnjenje

Dva digitalna filtera $H_0(z)$ i $H_1(z)$ su komplementarni po snazi ukoliko je suma kvadrata njihovih magnitudnih odziva jednaka 1 za svako ω , odnosno $|H_0(e^{j\omega})|^2 + |H_1(e^{j\omega})|^2 = 1$. Grafik sume mora biti ravna linija na vrijednosti 1, čime se vizuelno i računski potvrđuje ova osobina.

Zadatak 15: Uzorkovanje i aliasing u vremenskom domenu (Lab 8)

Dat je kontinualni signal oblika

$$x_a(t) = \cos(2\pi \cdot 1200 \cdot t) + \cos(2\pi \cdot 3600 \cdot t).$$

Napisati Scilab program koji:

1. generiše i prikazuje kontinualni signal $x_a(t)$ na intervalu $[0, 0.005]$ s korakom 0.00005,
2. generiše i prikazuje diskretnu verziju signala uzorkovanu sa periodom $T_1 = 0.0002$ s,
3. generiše i prikazuje diskretnu verziju signala uzorkovanu sa periodom $T_2 = 0.0005$ s,
4. za svaki period uzorkovanja komentarom u kodu naznačiti da li dolazi do aliasinga i zašto.

Rješenje

```
clf;

// Kontinualni signal (simuliran velikom frekvencijom uzorkovanja)
t = 0:0.00005:0.005;
f1 = 1200;
f2 = 3600;
xa = cos(2*%pi*f1*t) + cos(2*%pi*f2*t);

subplot(3,1,1);
plot(t, xa);
xlabel("Vrijeme [s]");
ylabel("Amplituda");
title("Kontinualni signal xa(t)");

// Uzorkovanje sa T1 = 0.0002 s => FT = 5000 Hz
// Nyquist brzina = 2 * 3600 = 7200 Hz
// FT = 5000 Hz < 7200 Hz => dolazi do aliasinga
T1 = 0.0002;
n1 = 0:T1:0.005;
xs1 = cos(2*%pi*f1*n1) + cos(2*%pi*f2*n1);
k1 = 0:length(n1)-1;

subplot(3,1,2);
plot2d3(k1, xs1);
scatter(k1, xs1, ".");
xlabel("Uzorak n");
ylabel("Amplituda");
title("Diskretni signal, T1=0.0002 s (FT=5000 Hz) - aliasing prisutan");

// Uzorkovanje sa T2 = 0.0005 s => FT = 2000 Hz
// FT = 2000 Hz < 7200 Hz => jaci aliasing, obje komponente aliasirane
T2 = 0.0005;
n2 = 0:T2:0.005;
```

```
xs2 = cos(2*%pi*f1*n2) + cos(2*%pi*f2*n2);  
k2 = 0:length(n2)-1;  
  
subplot(3,1,3);  
plot2d3(k2, xs2);  
scatter(k2, xs2, ".");  
xlabel("Uzorak n");  
ylabel("Amplituda");  
title("Diskretni signal, T2=0.0005 s (FT=2000 Hz) - jaci aliasing");
```

Objašnjenje: Nyquistova brzina za dati signal iznosi $2 \cdot 3600 = 7200$ Hz. Period $T_1 = 0.0002$ s odgovara frekvenciji uzorkovanja od 5000 Hz, što je manje od Nyquistove brzine, pa dolazi do aliasinga komponente f_2 . Period $T_2 = 0.0005$ s daje frekvenciju uzorkovanja od samo 2000 Hz, pa su aliasirane obje komponente signala.

Zadatak 16: Projektovanje i prikaz IIR filtera (Lab 8)

Napisati Scilab program koji projektuje i grafički poredi dva IIR lowpass filtera reda $N = 5$ sa normalizovanom cut-off frekvencijom $W_c = 0.3$:

1. Butterworth lowpass IIR filter,
2. Chebyshev tip 1 lowpass IIR filter sa dozvoljenom greškom $\delta_1 = 0.5$,
3. oba filtera prikazati na istom grafu u linearnom i logaritamskom (dB) mjerilu,
4. ispisati na ekran maksimalne vrijednosti magnitudnih odziva oba filtera.

Rješenje

```
clf;

// Parametri filtera
N = 5;
Wc = 0.3;
n = 256;

// Projektovanje Butterworth LP filtera
hz_butt = iir(N, 'lp', 'butt', Wc, []);
[hzm_butt, fr] = frmag(hz_butt, n);

// Projektovanje Chebyshev tip 1 LP filtera
hz_cheb = iir(N, 'lp', 'cheb1', Wc, [0.5 0.02]);
[hzm_cheb, fr] = frmag(hz_cheb, n);

// Ispis maksimalnih vrijednosti
disp("Maksimalna amplituda Butterworth filtera:");
disp(max(hzm_butt));
disp("Maksimalna amplituda Chebyshev tip 1 filtera:");
disp(max(hzm_cheb));

// Prikaz u linearnom mjerilu
subplot(2,1,1);
plot(fr', hzm_butt', 'b');
plot(fr', hzm_cheb', 'r');
xlabel("Normalizovana frekvencija");
ylabel("Amplituda");
title("Magnitudni odziv IIR filtera - linearno mjerilo");
legend("Butterworth", "Chebyshev tip 1");
mtlb_grid;

// Prikaz u logaritamskom mjerilu (dB)
subplot(2,1,2);
plot(fr', 20*log10(hzm_butt' + 1e-10), 'b');
plot(fr', 20*log10(hzm_cheb' + 1e-10), 'r');
xlabel("Normalizovana frekvencija");
ylabel("Pojacanje [dB]");
title("Magnitudni odziv IIR filtera - logaritamsko mjerilo");
```

```
legend("Butterworth", "Chebyshev tip 1");  
mtlb_grid;
```

Objašnjenje: Funkcija `iir` projektuje digitalni IIR filter metodom bilinearne transformacije. Butterworth filter ima maksimalno ravnu amplitudu u propusnom opsegu, dok Chebyshev tip 1 dozvoljava valovitost (*ripple*) u propusnom opsegu ali zauzvrat ima oštiji pad u prelaznom opsegu. Logaritamski prikaz u dB je pogodan za uočavanje ponašanja filtera u nepropusnom opsegu.

Zadatak 17: RGB, grayscale i binarna slika (Lab 9)

Napisati Scilab program koji koristeći IPCV toolbox radi sljedeće:

1. Učitati sliku `res/zadatak1/ulaz.jpg` koristeći `imread`, konvertovati je u grayscale pomoću `rgb2gray`, te prikazati originalnu i grayscale sliku jednu pored druge koristeći `subplot`.
2. Za dobijenu grayscale sliku kreirati binarnu sliku koristeći `im2bw`. Prag za binarizaciju odabrati na osnovu histograma slike (`imhist`), te prikazati binarnu sliku u trećem subplotu.
3. Implementirati vlastitu funkciju:

```
function Iout = ukloni_kanal(Img, kanal)
    // Img    - ulazna RGB slika
    // kanal  - indeks kanala koji se uklanja (1=R, 2=G, 3=B)
    // Iout   - izlazna slika sa nuliranim odabranim kanalom
```

Pozvati funkciju kako bi se uklonio crveni kanal slike, te rezultat prikazati u četvrtom subplotu.

Rješenje

```
clf;

// Ucitavanje slike
putanja = uigetfile('*.jpg', 'Odaberite sliku:');
I = imread(putanja);

// 1. Konverzija u grayscale i prikaz
Igray = rgb2gray(I);

subplot(2,2,1);
imshow(I);
title("Originalna RGB slika");

subplot(2,2,2);
imshow(Igray);
title("Grayscale slika");

// 2. Binarizacija na osnovu histograma
// Prikazati histogram kako bi se odabrao prag
imhist(Igray, [], 1);

// Prag odabran iz histograma, npr. 120/255 ~ 0.47
prag = 120 / 255;
Ibin = im2bw(Igray, prag);

subplot(2,2,3);
imshow(Ibin);
title("Binarna slika (prag = 120)");
```

```
// 3. Vlastita funkcija za uklanjanje kanala
function Iout = ukloni_kanal(Img, kanal)
    Iout = Img;
    Iout(:,:,kanal) = 0;
endfunction

Ibez_crvene = ukloni_kanal(I, 1);

subplot(2,2,4);
imshow(Ibez_crvene);
title("Slika bez crvenog kanala");
```

Objašnjenje: Funkcija `rgb2gray` konvertuje RGB sliku u grayscale koristeći ponderisanu sumu kanala. Prag za binarizaciju se bira iz histograma kao vrijednost koja sliku dijeli na dva jasno odvojena skupa piksela. Vlastita funkcija `ukloni_kanal` pristupa odgovarajućem sloju trodimenzionalne matrice slike i postavlja sve vrijednosti tog sloja na 0.

Zadatak 18: Filtriranje slike i uklanjanje šuma (Lab 10)

Napisati Scilab program koji koristeći IPCV toolbox radi sljedeće:

1. Učitati proizvoljnu sliku, konvertovati je u grayscale, te na nju dodati *salt-and-pepper* šum koristeći:

```
Izasumljena = imnoise(Igray, 'salt & pepper', 0.05);
```

2. Na zašumljenu sliku primijeniti **filter prosjeka** dimenzija 3×3 i **median filter** dimenzija 3×3 . Prikazati originalnu sliku, zašumljenu sliku i oba rezultata filtriranja u četiri subplota.

3. Implementirati vlastitu funkciju:

```
function Iout = filter_prosjeka(Igray, vel)
    // Igray - ulazna grayscale slika
    // vel    - dimenzija kvadratne maske (npr. 3 za 3x3)
    // Iout   - filtrirana slika
```

Funkcija treba manuelno, piksel po piksel (for petlja), računati prosječnu vrijednost okoline i dodijeliti je trenutnom pikselu. Testirati funkciju na zašumljenoj slici i prikazati rezultat.

Rješenje

```
clf;

// Ucitavanje i konverzija u grayscale
putanja = uigetfile('*..*', 'Odaberite sliku:');
I = imread(putanja);
Igray = rgb2gray(I);

// 1. Dodavanje salt-and-pepper suma
Izasumljena = imnoise(Igray, 'salt & pepper', 0.05);

// 2. Filter prosjeka 3x3 (koristeci IPCV)
maska = ones(3, 3) / 9;
I_avg = imconv(double(Izasumljena), maska);
I_avg = uint8(I_avg);

// Median filter 3x3
I_med = immedian(Izasumljena, 3, 3);

// Prikaz rezultata
subplot(2,2,1);
imshow(Igray);
title("Originalna slika");

subplot(2,2,2);
imshow(Izasumljena);
title("Zasumljena slika (s&p, 0.05)");
```



```

subplot(2,2,3);
imshow(I_avg);
title("Filter prosjeka 3x3");

subplot(2,2,4);
imshow(I_med);
title("Median filter 3x3");

// 3. Vlastita implementacija filtera prosjeka
function Iout = filter_prosjeka(Igray, vel)
    Igray = double(Igray);
    [r, c] = size(Igray);
    Iout = Igray;
    offset = floor(vel / 2);

    for i = 1+offset : r-offset
        for j = 1+offset : c-offset
            okolina = Igray(i-offset:i+offset, j-offset:j+offset);
            Iout(i, j) = mean(okolina(:));
        end
    end

    Iout = uint8(Iout);
endfunction

I_vlastiti = filter_prosjeka(Izasumljena, 3);

scf();
subplot(1,2,1);
imshow(Izasumljena);
title("Zasumljena slika");

subplot(1,2,2);
imshow(I_vlastiti);
title("Vlastiti filter prosjeka 3x3");

```

Objašnjenje: Filter prosjeka zamjenjuje svaki piksel prosječnom vrijednošću njegove okoline, čime se uklanjaju nagle promjene intenziteta nastale šumom. Median filter je bolji za *salt-and-pepper* šum jer ne uvodi nove vrijednosti već bira stvarnu vrijednost iz okoline, pa ivice ostaju očuvanije. Vlastita implementacija filtera prosjeka koristi ugnježdene **for** petlje kojima se prolazi kroz svaki piksel i računa srednja vrijednost kvadratne okoline zadane veličine **vel**.